Analysis and Design of Algorithms

NUS National University of Singapore

Algorithms

CS3230

Tutorial

Week 12

Question 1

Let L and L' be decision problems in NP. Select all true statements.

(I) If $L \leq_P L'$ and L is NP-complete, then L' is NP-complete.

(II) If $L \leq_P L'$ and L' is NP-complete, then L is NP-complete.

(III) If $L \leq_P L'$ and L is in P, then L' is in P.

(IV) If $L \leq_P L'$ and L' is in P, then L is in P.

- (I) only
- (II) only
- (I) and (II) only
- (II) and (III) only
- (I) and (IV) only

Question 1

Answer: E (I) and (IV) only

$$L \leq_P L'$$

(I) True: L NP-complete → L' NP-complete

L' can be used to solve L, so is as hard as L

(II) False: L' NP-complete → L NP-complete

(Common mistake, please do not make it)

All problems in NP poly time reducible to a NP-complete problem, yet some are in P.

Question 1

Answer: E (I) and (IV) only

$$L \leq_P L'$$

(III) False: L in P → L' in P

All problems in P poly time reducible to a NP-complete problem, so L' may not be in P

(IV) True: L' in P → L in P

L' can be used to solve L, so can solve L in poly time by doing the reduction then solving L'.

Question 2

Consider the two problems:

PARTITION: Given a set of positive integers S, can the set be partitioned into two sets of equal total sum?

BALL-PARTITION: Given k balls, can we divide the balls into two boxes with an equal number of balls?

We try to show that PARTITION $\leq_P$ BALL-PARTITION using the following transformation A:

1) From the problem PARTITION, we are given S, a set of positive integers.

2) We define k as the total sum of all the elements in S.

3) We use this number k for the BALL-PARTITION problem.

What is wrong with this transformation?

- The transformation is correct.
- A YES solution to $A(S)$ does not imply a YES solution to S.
- A YES solution to S does not imply a YES solution to $A(S)$
- The transformation does not run in polynomial time.

Question 2

Answer: B

Option A: Option is wrong.

The transformation is wrong. See part B

Question 2

Answer: B

Option B: Option is correct.

A YES solution to A(S) does not imply a YES solution to S.

A(S): BALL-PARTITION. **S:** PARTITION

S = {1,3}. Converted to A(S) = 4

{1,3} cannot be Partitioned,

But 4 balls can be “Ball-Partitioned” (into 2,2)

Question 2

Answer: B
Option C: Option is wrong.

If the answer to the PARTITION is YES, we can set the number of balls in each box in BALL-PARTITION to the sum in each set to get equal number of balls in each box.

Question 2

Answer: B
Option D: Option is wrong.

Just need to sum the numbers in the set for the transformation, so it's poly time.

Question 3

PARTITION problem: Given a set T of nonnegative integers, can we partition T into two sets of equal total sum?

KNAPSACK problem: Given n items described by non-negative integer pairs (w_1,v_1), ... (w_n,v_n), capacity W and threshold V, is there a subset of item with total weight at most W and total value at least V?

PARTITION instances with total weights that are odd cannot be partitioned into two equal weight sets, hence can immediately be answered with a NO answer. Consider the following transformation of PARTITION instances with total weights that are even numbers into instances of KNAPSACK:

Given a PARTITION instance $\{w_1, ..., w_n\}$ with total sum $S = \sum_{i=1}^{n} w_i$, construct a KNAPSACK instance (w_1,w_1), ... (w_n,w_n) with capacity $W = S/2$ and threshold $V = S/2$.

Which of the following are true?

(I) A YES answer to the PARTITION instance implies a YES answer to the KNAPSACK instance.

(II) A YES answer to the KNAPSACK instance implies a YES answer to the PARTITION instance

(III) The transformation runs in polynomial time

(IV) Since PARTITION instances can be transformed into KNAPSACK instances in polynomial time and KNAPSACK is a NP-complete problem, PARTITION is NP-hard.

- (I) and (III) only
- (II) and (III) only
- (I), (II), and (III) only
- All are correct.

Question 3

Answer: C (I), (II), (III) only

- **(I)** YES answer to PARTITION implies YES answer to KNAPSACK:

Simply use the same subset in PARTITION for knapsack -- will get weight S/2 and value S/2

Question 3

Answer: C (I), (II), (III) only

- **(II)** YES answer to KNAPSACK implies YES answer to PARTITION:

YES answer to KNAPSACK means there is a subset with weight no more than S/2 with value at least S/2. Since weight equals value in the transformed instances, only way that can happen is if weight and value equal S/2. Same subset can be used for PARTITION.

Question 3

- **(III):** Transformation simply copies weight to value, so poly time.

- **(IV) Is not True**: We have shown that PARTITION $\leq_P$ KNAPSACK. To show that PARTITION is NP-hard, need to show KNAPSACK $\leq_P$ PARTITION.
  Note that PARTITION is indeed NP-hard, although not shown here.

Question 4

Your boss asks you to write a program to find the optimal solution to BestRoute, one of the crucial parts of the company's service that needs to be run every day. After trying a long time, you are not able to come up with a polynomial time algorithm to do it. Which of the following will convince your boss that he should not fire you and hire the hot-shot programmer from a competitor company? (Hint: your boss is an SoC alumnus who has taken CS3230 before.)

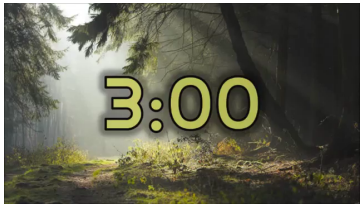
(I) Write an exhaustive search program for BestRoute and run it to show your boss how long it takes to solve the problem.

(II) Show that instances of BestRoute can be transformed into instances of Travelling Salesman in polynomial time.

(III) Show that instances of Travelling Salesman can be transformed into instances of BestRoute in polynomial time.

(IV) Show that instances of FACTOR can be transformed into instances of BestRoute in polynomial time.

- (I) only
- (I) and (II) only
- (III) only
- (III) and (IV) only



Question 4

Answer: D (III) and (IV) only

- (I) does not show hardness, only that your program is slow.
- (II) does not show hardness, only that TSP can be used to solve BestRoute
- (III) shows that if you can solve BestRoute in poly time, you can solve TSP in poly time. TSP is NP-hard, so BestRoute is also NP-hard.
- (IV) shows that if you solve BestRoute in poly time, you can solve FACTOR in poly time and hence can break RSA in poly time.